

Backtracking

Técnicas de Diseño de Algoritmos

FCEyN UBA

September 9, 2026



Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Backtracking

Idea: recorrer sistemáticamente todas las posibles configuraciones del espacio de soluciones de manera recursiva.

Tenemos:

- ▶ Soluciones parciales
- ▶ Soluciones candidatas
- ▶ Soluciones válidas

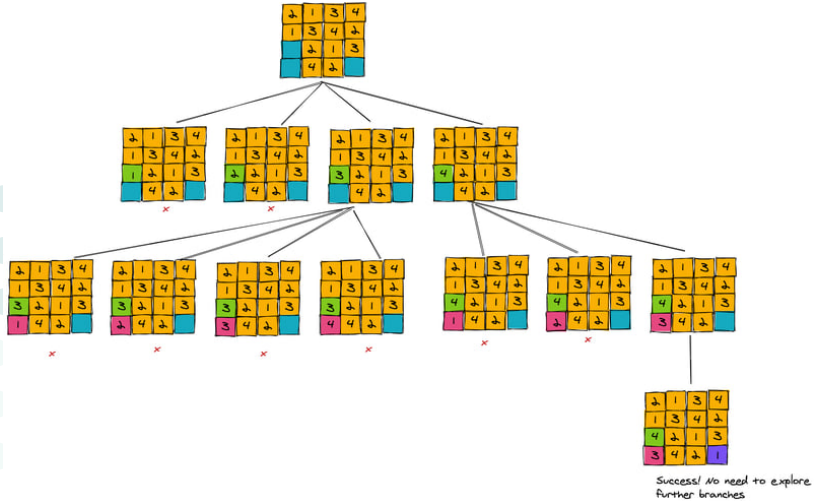
Sudoku

Tomemos como ejemplo un Sudoku de 4x4 en el que la regla es que no se pueden repetir números en filas ni en columnas. Queremos hacer un algoritmo de backtracking que lo resuelva:

- ▶ En cada casillero libre (azules) puede poner números del 1 al 4
- ▶ Como queremos evaluar todas las opciones posibles probamos con todas las combinaciones de números

2	1	3	4
1	3	4	2
	2	1	3
4	2		

Sudoku 4x4
Fill in the blue cells
Rule:
No number is repeated
in a row or column



Sudoku

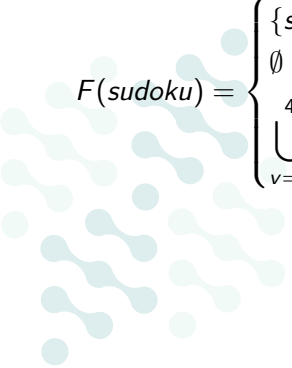
- ▶ Soluciones **parciales**: todos los tableros posibles a medida que los voy completando
- ▶ Soluciones **candidatas**: los tableros llenos, sean o no válidos (en este caso solo tenemos el válido)
- ▶ Soluciones **válidas**: el sudoku completado correctamente que cumple todas las reglas

Sudoku

En el dibujo lo que está marcado con cruces rojas lo llamamos podas. Una poda es cortar ramas del árbol que no nos llevan a soluciones válidas. En este caso “podamos” cuando el número que intentamos poner ya está ubicado en otro lugar de la misma fila o columna

Sudoku

Hagamos la función recursiva (sin la poda):


$$F(sudoku) = \begin{cases} \{sudoku\} & \text{si } sudoku \text{ está completo y es válido} \\ \emptyset & \text{si } sudoku \text{ está completo y es inválido} \\ \bigcup_{v=1}^4 F(sudoku \cup \{(i,j) \rightarrow v\}) & \text{si } (i,j) \text{ es la primera casilla vacía} \end{cases}$$

Sudoku

Sea n la cantidad de casilleros libres, ¿cuál es la complejidad de esta idea? ¿Podría ser mejor si agregamos más podas?

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Subset Sum

Subset Sum

Dado un array `arr[]` de enteros no negativos y un valor `sum`, la tarea es chequear si existe un subconjunto `sum` del array cuya suma dé exactamente `sum`.

1. Dar un algoritmo basado en Backtracking que resuelva el problema
2. Dibujar el árbol de backtracking
3. Dar la complejidad del algoritmo

Subset Sum: idea del algoritmo

Recorremos el array de izquierda a derecha y, en cada posición i , tomamos una decisión binaria:

- ▶ **Tomar** $arr[i]$: lo agrego a la solución parcial y le resto su valor a lo que falta sumar
- ▶ **No tomar** $arr[i]$: sigo con el resto del array sin cambiar lo que falta sumar

Nos alcanza con dos parámetros:

- ▶ i : la posición que estamos decidiendo
- ▶ s : cuánto falta para llegar a sum (es decir, sum menos lo ya elegido)

Subset Sum: casos base y podas

Casos base:

- ▶ Si $s = 0$: la solución parcial ya suma $sum \Rightarrow \text{true}$
- ▶ Si $i = n$ y $s \neq 0$: me quedé sin elementos para decidir $\Rightarrow \text{false}$

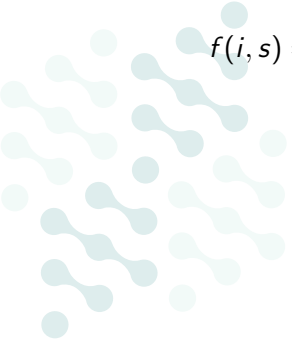
Podas:

- ▶ Si $s < 0$: me pasé. Como **los elementos son no negativos**, agregar más nunca va a bajar la suma
- ▶ Si $\sum_{k=i}^{n-1} arr[k] < s$: aunque tome todo lo que queda no llego

Además, apenas encontramos un subconjunto válido podemos cortar y no seguir explorando (el or corta solo).

Subset Sum: función recursiva

Sea n el tamaño del array:


$$f(i, s) = \begin{cases} \text{true} & \text{si } s = 0 \\ \text{false} & \text{si } i = n \text{ y } s \neq 0 \\ f(i + 1, s - arr[i]) \vee f(i + 1, s) & \text{si no} \end{cases}$$

Subset Sum: pseudocódigo

```
subsetSum(arr, n, i, s):  
    if s == 0:  
        return true  
    if i == n or s < 0:  
        return false  
    if suffixSum(arr, i) < s:  
        return false  
    return subsetSum(arr, n, i + 1, s - arr[i]) or  
           subsetSum(arr, n, i + 1, s)
```

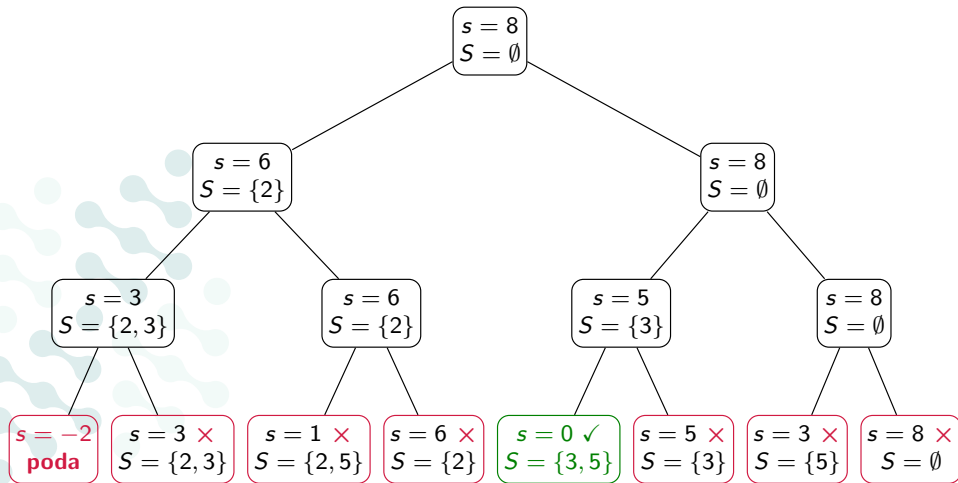
Se llama con subsetSum(arr, n, 0, sum).

Subset Sum: árbol de backtracking

$arr = [2, 3, 5]$, $sum = 8$. Cada nodo tiene el s que falta y la solución parcial S .

Rama izquierda: **tomo** $arr[i]$

Rama derecha: **no tomo** $arr[i]$



Subset Sum: complejidad

El árbol es binario y tiene un nivel por cada elemento del array:

- ▶ 2^n hojas (una por cada subconjunto posible), $O(2^n)$ nodos en total*
- ▶ En cada nodo hacemos trabajo $O(1)$

Entonces la complejidad es $O(2^n)$.

* Pues hay $2^0 + 2^1 + \dots + 2^{n-1} = 2^n - 1$ nodos que no son hojas

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: **MaxiSubconjunto**

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Problema: MaxiSubconjunto

Dada una matriz simétrica M de $n \times n$ números naturales y un número k , queremos encontrar un subconjunto I de $\{1, \dots, n\}$ con $|I| = k$ que maximice $\sum_{i,j \in I} M_{ij}$. Los índices de las filas y columnas empiezan en 1. Por ejemplo, si $k := 3$ y

$$M := \begin{pmatrix} 0 & 10 & 10 & 1 \\ 10 & 0 & 5 & 2 \\ 10 & 5 & 0 & 1 \\ 1 & 2 & 1 & 0 \end{pmatrix},$$

entonces $I := \{1, 2, 3\}$ es una solución óptima, cuyo valor es $10 + 10 + 10 + 5 + 10 + 5 = 50$.

MaxiSubconjunto

1. Dar una propuesta de conjunto de soluciones candidatas y predicado de validación $valida_{M,k}(a)$ con $a \in \text{candidatas}$ para este problema.
2. Dar un algoritmo basado en Backtracking que resuelva el problema
3. Dar la complejidad del algoritmo
4. Proponer una poda por optimalidad

MaxiSubconjunto

Dada una matriz simétrica M de $n \times n$ números naturales y un número k , encontrar un subconjunto I de $\{1, \dots, n\}$ con $|I| = k$ que maximice $\sum_{i,j \in I} M_{ij}$.

1. Dar una propuesta de conjunto de soluciones candidatas y predicado de validación $valida_{M,k}(a)$ con $a \in \text{candidatas}$ para este problema.
2. Dar un algoritmo basado en Backtracking que resuelva el problema
3. Dar la complejidad del algoritmo
4. Proponer una poda por optimalidad

1. Candidatas y validación

Una candidata es cualquier subconjunto de $\{1, \dots, n\}$:

$$candidatas = \{ I : I \subseteq \{1, \dots, n\} \}$$

Es válida si tiene exactamente k elementos:

$$valida_{M,k}(I) = (|I| = k)$$

Ojo: M no aparece en la validación. No dice *cuáles* subconjuntos se pueden elegir, sólo *cuánto vale* cada uno.

2. La idea

Recorremos los elementos $1, 2, \dots, n$ en orden. De cada uno decidimos una sola cosa: **entra o no entra**.

Una *solución parcial* es entonces:

- ▶ i : el próximo elemento a decidir,
- ▶ I : los que ya metimos (todos menores que i).

De (i, I) salen dos ramas:

$$(i + 1, I) \quad \text{y} \quad (i + 1, I \cup \{i\})$$

El árbol es binario y tiene profundidad n .

2. La función recursiva

Llamamos $z(I) = \sum_{i,j \in I} M_{ij}$ al valor de un subconjunto.

$$ms(i, I) = \begin{cases} z(I) & \text{si } i > n \text{ y } |I| = k \\ -\infty & \text{si } i > n \text{ y } |I| \neq k \\ \max(ms(i+1, I), ms(i+1, I \cup \{i\})) & \text{si no} \end{cases}$$

La respuesta es $ms(1, \emptyset)$.

Las hojas son todos los subconjuntos posibles. Las que no tienen k elementos valen $-\infty$, así que el max las descarta solo.

3. Complejidad

El árbol es binario de profundidad n , así que tiene $O(2^n)$ nodos.

En cada hoja calculamos $z(I)$, que son $|I|^2 \leq k^2$ sumas:

$$\text{Tiempo} = O(2^n k^2)$$

$$\text{Espacio} = O(n)$$

la pila de recursión, más el conjunto I que vamos agrandando y achicando.

Mejora: en vez de calcular $z(I)$ de cero en cada hoja, se puede ir arrastrando el valor y actualizarlo en $O(k)$ al agregar un elemento. Queda $O(2^n k)$.

4. Poda por optimalidad

Guardamos $z^* =$ el mejor valor que encontramos hasta ahora.

Sea μ la entrada más grande de M . Estamos en (i, I) y queremos saber cuánto *como máximo* puede llegar a valer una solución que extienda a I .

La solución final va a tener k elementos, o sea k^2 pares. De esos, los $|I|^2$ pares de I ya los tenemos contados en $z(I)$. Los que faltan son $k^2 - |I|^2$, y cada uno vale a lo sumo μ :

$$z(\text{cualquier extensión de } I) \leq z(I) + \mu (k^2 - |I|^2)$$

4. Poda por optimalidad

$$z(\text{cualquier extensión de } I) \leq z(I) + \mu (k^2 - |I|^2)$$

Entonces podemos podar (i, I) cuando

$$z(I) + \mu (k^2 - |I|^2) \leq z^*$$

¿Por qué es correcta? Si se cumple, todo lo que cuelga de esa rama vale a lo sumo z^* . Y ya tenemos en la mano una solución que vale z^* . Así que no perdemos ningún óptimo.

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

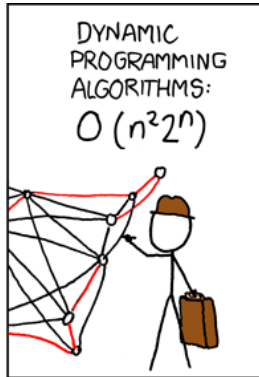
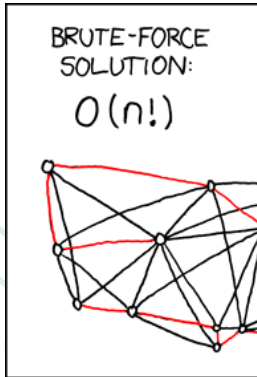
Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Viajante de comercio



Viajante de comercio

Viajante de comercio

Sea $G = (V, E)$ un grafo completo (es decir, donde hay aristas entre todo par de nodos) con una función de peso $w : E \rightarrow \mathbb{N}$, que puede interpretarse como el costo de moverse de un nodo a otro. El problema del *viajante de comercio* consiste en encontrar un recorrido que pase por todos los nodos una sola vez de manera de minimizar la suma de los costos, y luego vuelva al primer nodo. En otras palabras, se busca encontrar una permutación π de $\{1, \dots, n\}$ (el conjunto de nodos) que minimice $w(\pi(n), \pi(1)) + \sum_{i=1}^{n-1} w(\pi(i), \pi(i+1))$.

Viajante de comercio

Hay que minimizar $w(\pi(n), \pi(1)) + \sum_{i=1}^{n-1} w(\pi(i), \pi(i+1))$.

Por ejemplo, si representamos a w como una matriz

$$w = \begin{pmatrix} 0 & 1 & 10 & 10 \\ 10 & 0 & 3 & 15 \\ 21 & 17 & 0 & 2 \\ 3 & 22 & 30 & 0 \end{pmatrix},$$

entonces $\pi(i) = i$ es una solución óptima.

Viajante de comercio

1. Diseñar un algoritmo de backtracking para resolver el problema. Demostrar que es correcto indicando claramente cómo se codifica una solución candidata, cuáles soluciones son válidas y cómo están ordenadas, qué es una solución parcial, cómo se extiende cada solución parcial, y la semántica de la función recursiva.
2. Calcular la complejidad temporal y espacial del mismo.
3. Proponer una poda por optimalidad y mostrar que es correcta.

Viajante de comercio - Soluciones

- ▶ Conjunto de soluciones candidatas: permutaciones de $\{1, \dots, n\}$
 $\{(\pi_1, \dots, \pi_n) : 1 \leq \pi_i \leq n \wedge \text{sinRepetidos}(\pi)\}$
- ▶ Conjunto de soluciones válidas: mismo conjunto de soluciones candidatas
(válida(π) = True)
- ▶ Función a optimizar: minimizar
 $\text{valor}(\pi) = w(\pi(n), \pi(1)) + \sum_{i=1}^{n-1} w(\pi(i), \pi(i+1))$
- ▶ Soluciones parciales: caminos simples de largo menor o igual a n (incluyendo el camino vacío, donde arrancamos la recursión)
 $\{(\pi_1, \dots, \pi_k) : 0 \leq k \leq n \wedge 1 \leq \pi_i \leq n \wedge \text{sinRepetidos}(\pi)\}$

Viajante de comercio - Backtracking

- ▶ Soluciones parciales: caminos simples de largo menor o igual a n , **¿cómo podemos extender una solución parcial?**
- ▶ Las sucesoras de la solución parcial π surgen de agregar un nuevo nodo al camino, respetando que no se pueden repetir: $\pi \oplus v$ para $1 \leq v \leq n, v \notin \pi$.

$$viajante(\pi) = \begin{cases} valor(\pi) & \text{si } |\pi| = n \\ \min\{viajante(\pi \oplus v) : 1 \leq v \leq n \wedge v \notin \pi\} & \text{caso contrario} \end{cases}$$

Solución al problema: $viajante([])$.

Nota: estamos asumiendo que w es accesible globalmente (no lo pasamos como parámetro).

¿Y el camino? Veremos cómo conseguirlo más adelante.

Viajante de comercio - Complejidad

$$viajante(\pi) = \begin{cases} valor(\pi) & \text{si } |\pi| = n \\ \min\{viajante(\pi \oplus v) : 1 \leq v \leq n \wedge v \notin \pi\} & \text{caso contrario} \end{cases}$$

Caso base: $O(n)$.

Caso recursivo: depende cómo chequeamos $v \notin \pi$:

- ▶ Si iteramos todos los $1 \leq v \leq n$ y por cada uno chequeamos $v \notin \pi$, el chequeo es $O(n)$ y en total queda $O(n^2)$ (más el costo de las recursiones).
- ▶ Una idea mejor: arreglo binario de visitados.
 1. Armamos un arreglo binario de visitados, recorriendo π y marcando en True los nodos que estén en π : $O(n)$ en peor caso.
 2. Cuando quiera chequear $v \notin \pi$, lo hago en ese arreglo en $O(1)$.
 3. Costo final: $O(n)$ (más el costo de las recursiones). Problema: uso $O(n)$ memoria en cada llamado.
- ▶ Tener un arreglo de visitados, ir modificándolo y pasándolo. Considerando que lo pasamos por referencia (al igual que π) podemos implementar con $O(1)$ memoria por llamado.

Viajante de comercio - Complejidad

$$\text{viajante}(\pi, \text{visitados}) = \begin{cases} \text{valor}(\pi) & \text{si } |\pi| = n \\ \min\{\text{viajante}(\pi \oplus v, \text{visitados}[v] \leftarrow \text{True}) : & \text{caso contrario} \\ 1 \leq v \leq n \wedge \neg \text{visitados}[v]\} \end{cases}$$

¿Cómo mejoramos el $O(n)$ del caso base? Podemos ir calculando la suma.

Viajante de comercio - Complejidad

$$viajante(\pi, visitados, suma) = \begin{cases} suma & \text{si } |\pi| = n \\ \min\{viajante(\pi \oplus v, visitados[v] \leftarrow True, \\ suma') : 1 \leq v \leq n \wedge \neg visitados[v]\} & \text{caso contrario} \end{cases}$$

donde

$$suma' = \begin{cases} suma & \text{si } |\pi| = 0 \\ suma + w(ultimo(\pi), v) + w(v, \pi[0]) & \text{si } |\pi \oplus v| = n \\ suma + w(ultimo(\pi), v) & \text{caso contrario} \end{cases}$$

Viajante de comercio - Correctitud

Queremos ver que

$$viajante(\pi) = \begin{cases} valor(\pi) & \text{si } |\pi| = n \\ \min\{viajante(\pi \oplus v) : 1 \leq v \leq n \wedge v \notin \pi\} & \text{caso contrario} \end{cases}$$

consigue el costo mínimo para nuestro problema.

Primero demostremos un lema que nos va a ser útil.

Viajante de comercio - Correctitud

Lema: Sea $T(\pi)$ el costo del camino mínimo que usa a π de prefijo con $|\pi| < n$. Entonces $T(\pi)$ es igual al mínimo entre los costos de todos los caminos que podemos armar extendiendo a π en un elemento que no está en él: $T(\pi) = \min_{v \notin \pi} T(\pi \oplus v)$.

Dem: Consideremos el camino de costo mínimo que usa a π de prefijo, digamos que es π^* , este camino coincide π en los primeros $|\pi|$ elementos y luego tiene un nodo v en la $|\pi| + 1$ posición donde v no pertenece a π . Entonces π^* pertenece al conjunto de todos los caminos que tiene de prefijo a π y que lo extienden con un elemento más que no está en él. Ahora si tomamos el mínimo sobre todos los caminos que extienden a π tenemos

$$T(\pi) \leq \min_{v \notin \pi} T(\pi \oplus v)$$

Pero además no existe uno de costo más pequeño que el de π^* y como este pertenece al conjunto sobre el que estamos minimizando sale que

$$\min_{v \notin \pi} T(\pi \oplus v) \leq T(\pi)$$

Viajante de comercio - Correctitud

Ahora vamos a probar que la función *viajante* coincide con T para todo prefijo de camino, es decir

$$\text{viajante}(\pi) = T(\pi)$$

. Para esto vamos a hacer inducción en

$$r = n - |\pi|$$

.
Caso base: $r = 0 = n - |\pi|$, el camino tiene longitud n por lo cual *viajante*(π) cae en el caso base de su recursión y es igual a *valor*(π), por otro lado $T(\pi) = \text{valor}(\pi)$ ya que el camino no se puede extender más. Entonces $\text{viajante}(\pi) = T(\pi)$.

Viajante de comercio - Correctitud

Caso inductivo: Supongamos que vale para $r = n - |\pi'|$, es decir que dado un π' de longitud r tenemos que $\text{viajante}(\pi') = T(\pi')$ que es el costo mínimo del camino con prefijo π' . Veamos que pasa para $r + 1$.

Sea π un camino parcial tal que $r + 1 = n - |\pi|$. Si lo evaluamos en *viajante* caemos en el segundo caso de la recursión y tenemos

$$\text{viajante}(\pi) = \min\{\text{viajante}(\pi \oplus v) : 1 \leq v \leq n \wedge v \notin \pi\}$$

Ahora si miramos la parte derecha de la expresión, $\pi \oplus v$ tiene tamaño $|\pi| + 1$ por lo cual $n - |\pi| + 1 = r$ y estamos en condiciones de aplicar la HI. Tenemos

$$\text{viajante}(\pi) = \min_{v \notin \pi}\{\text{viajante}(\pi \oplus v)\} = \min_{v \notin \pi} T(\pi \oplus v) = T(\pi)$$

Que es justamente lo que queríamos demostrar. Ahora para cerrar la demostración notemos que $\text{viajante}(\emptyset) = T(\emptyset)$ y esto es justamente el costo del camino mínimo que tiene un prefijo de camino vacío, es decir es el costo del camino mínimo que podemos armar con elementos de V .

Viajante de comercio - Código

```
def viajante(camino, visitados, suma):
    if len(camino) == n:
        return suma
    else:
        mejor = INF
        for v in range(1, n+1):
            if not visitados[v-1]:
                # Preparo los parametros de la recursion
                visitados[v-1] = True
                if len(camino) == 0:
                    suma_nueva = suma
                elif len(camino) == n-1:
                    suma_nueva = suma + w[camino[-1]-1][v-1] + w[v-1][camino[0]-1]
                else:
                    suma_nueva = suma + w[camino[-1]-1][v-1]
                camino.append(v)
                # Llamada recursiva
                rec = viajante(camino, visitados, suma_nueva)
                mejor = min(mejor, rec)
                # Deshago cambios en camino y visitados
                camino.pop()
                visitados[v-1] = False
        return mejor
```

Complejidad temporal: $T(m) = m \cdot T(m-1) + O(m) = O(m!)$ donde $m = n - |\pi|$ vértices por decidir. **Complejidad espacial:** $O(n)$. La profundidad del árbol es n y cada llamado usa $O(1)$ memoria.

Viajante de comercio - Comentarios finales

¿Y el camino?

- ▶ Una opción incómoda pero limpia matemáticamente es devolver una tupla (*mejor_costo*, *mejor_camino*).
- ▶ Una opción cómoda pero sucia es tener variables globales *mejor_costo* y *mejor_camino*. Esta opción permite una poda por optimalidad muy común en problemas de minimización: si mi *suma* es mayor a *mejor_costo*, podo. Se puede hacer sin variables globales pero es molesto.

Notar que si el problema era encontrar el valor mínimo, en vez del camino mínimo, **el conjunto de soluciones NO cambia**. Sigue siendo el mismo problema, solo que nos piden devolver el valor en vez de la solución.

Si les gustó este problema, cursen Investigación Operativa.

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

Palabras en cadena

Palabras en cadena

Dada una cadena de letras sin espacios o puntos queremos analizar si se puede subdividir de forma de obtener todas palabras válidas.

- ▶ Dar una función recursiva que resuelva el problema.
- ▶ Calcular una cota superior para la complejidad de implementar esa función con backtracking.
- ▶ Demostrar que el algoritmo es correcto, definiendo los conjuntos de soluciones, las relaciones entre soluciones, y las semánticas que sean necesarios.

Palabras en cadena - Ejemplo

Ejemplo:

TDAeslamejormateriadelDC

Se puede subdividir en³

TDA|es|la|mejor|materia|del|DC

Mientras que

nosvemoc

No tiene una subdivisión válida.

³Asumimos que TDA y DC están en nuestro diccionario.

Palabras en cadena - Función Recursiva

Idea:

- ▶ Si $S[0, i]$ es una palabra verificar que $S[i + 1, |S|]$ se puede separar
- ▶ Si $S[0, i]$ **no** es una palabra probar con otro $i + 1$

Palabras en cadena - Función Recursiva

Caso base:

- Si $|S| = 0$ devolvemos **true**

Caso recursivo:

- Si $|S| > 0$ probamos $\bigvee_{i=1}^{|S|} \text{palabra}(S[0, i]) \wedge f(S[i + 1, |S|])$

Resultado final:

$$f(S) = \begin{cases} \text{true} & \text{si } |S| = 0 \\ \bigvee_{i=1}^{|S|} \text{palabra}(S[0, i]) \wedge f(S[i + 1, |S|]) & \text{si } |S| > 0 \end{cases}$$

Palabras en cadena - Complejidad

Analizamos la recurrencia asumiendo que *palabra* tiene una complejidad de $O(|c|)$ para una entrada c

$$T(n) = \sum_{k=0}^{n-1} T(k) + O(n)$$

$$T(n-1) = \sum_{k=0}^{n-2} T(k) + O(n-1)$$

Objetivo: $T(n)$ debe depender solamente de $T(n-1)$

Palabras en cadena - Complejidad

Vamos a sobreaproimar la recurrencia reemplazando $O(n)$ por una constante c de la definición de O .

Luego comparamos las recurrencias $T(n)$ y $T(n - 1)$:


$$\begin{aligned}T(n) - T(n - 1) &= \sum_{k=0}^{n-1} T(k) + c - \left[\sum_{k=0}^{n-2} T(k) + c(n - 1) \right] \\&= T(n - 1) + \sum_{k=0}^{n-2} T(k) + c - \sum_{k=0}^{n-2} T(k) - c \\&= T(n - 1) + c \\T(n) &= 2T(n - 1) + c\end{aligned}$$

Palabras en cadena - Complejidad

Expandiendo el resultado anterior tenemos que

$$\begin{aligned}T(n) &= 2T(n-1) + c \\&= 2(2T(n-2) + c) + c = 2^2 T(n-2) + c \\&= 2^3 T(n-3) + c \\&= 2^4 T(n-4) + c \\&= \dots\end{aligned}$$

Luego tenemos que

$$T(n) = 2^k T(n-k) + c$$

y tomando $n = k$

$$T(n) = O(\text{costo}(\text{palabra}(n))2^n)$$

Palabras en cadena - Demostración

Queremos probar que $f(S)$ computa correctamente si para toda cadena S de longitud n podemos separarla en palabras o no.

$P(n) = \forall S: \text{cadena}, |S| = n, f(S)$ nos dice si podemos separar S en palabras o no.

Caso base:

$P(0)$, tenemos que probar que vale para toda cadena de longitud 0. Como no podemos subdividir más la cadena deberíamos devolver True que es exactamente lo que devuelve nuestra función recursiva si le damos una cadena vacía.

Palabras en cadena - Demostración

Paso inductivo: En este caso, $|S| > 0$, con lo cual vamos a separar a S en 2 subcadenas de longitud k y $n - k$ para $k = 1 \dots n$. Vamos a tener 2 casos:

Caso S subdividible: Como es subdividible, existe un k tal que $S[0, k]$ es palabra. Mientras que la otra subcadena, la vamos a validar con $f(S[k + 1, n])$ donde $|S[k + 1, n]| < n$, luego, por **HI** sabemos que f devuelve true al sufijo de S .

Palabras en cadena - Demostración

Caso S no admite subdivisión: Vamos a tener 2 subcasos más:

- ▶ No existe k tal que $palabra(S[0, k])$ es true.
- ▶ Existe k tal que $palabra(S[0, k])$ es true, con lo cual debe suceder que $f(S[k + 1, n])$ sea false. Como $|S[k + 1, n]| < n$ sabemos por **HI** que f funciona correctamente, luego $f(S[k + 1, n])$ es false.

Por lo tanto, como probamos ambos casos del algoritmo por inducción sabemos que es correcto.

Lo que veremos hoy

Introducción y motivación

Ejemplo: sudoku

Ejercicio 1: Suma Subconjuntos

Ejercicio 3: MaxiSubconjunto

Ejercicio 14: Viajante de comercio

Ejercicio 15: Palabras en cadena

Ejercicio 17: ABB óptimo

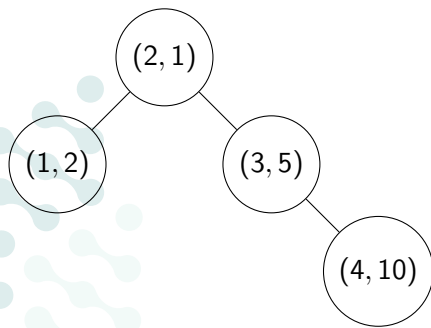
ABB Óptimo

Dado un conjunto de elementos de $[n] = \{1, \dots, n\}$, y una función $f: [n] \rightarrow \mathbb{N}$ que nos da la frecuencia de acceso a dichos elementos, decimos que A es un árbol binario de búsqueda óptimo si este minimiza el costo de todos los accesos dados por f .

En otras palabras el costo de acceder a un elemento i es la cantidad de nodos del árbol que hay que recorrer desde la raíz para llegar a i , incluyendo la raíz e i .

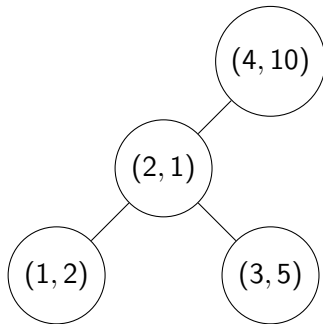
ABB óptimo

Ejemplo: Si $n = 4$ y $f(1) = 2, f(2) = 1, f(3) = 5, f(4) = 10$ y si notamos a los vértices como $(v, f(v))$ tenemos el siguiente árbol balanceado de búsqueda y un ABB óptimo bajo nuestra métrica:



Costo:

$$1 \cdot 1 + 2 \cdot 2 + 5 \cdot 2 + 10 \cdot 3 = 45$$



Costo:

$$10 \cdot 1 + 1 \cdot 2 + 2 \cdot 3 + 5 \cdot 3 = 33$$

ABB óptimo

1. Escribir una función recursiva que devuelva el costo de acceder a todos los elementos de un ABB dado usando f .
2. Escribir un algoritmo de backtracking que encuentre el ABB óptimo para un f dado.
3. Dar una cota superior para la complejidad (Ayuda: pasar de la función recursiva a una recurrencia que solo dependa del tamaño de la entrada).
4. Probar que el algoritmo es correcto.

ABB óptimo - Costo de un ABB

1. Escribir una función recursiva que devuelva el costo de acceder a todos los elementos de un ABB dado usando f .

Sabemos, por definición, que el costo de un ABB A con frecuencias f es:

$$\text{costo}(A, f) = \sum_{i=1}^{|f|} f[i] \cdot \text{profundidad}(i, A)$$

Sea r la raíz de A .

Obs. 1: Como A es un ABB, todos los elementos $i < r$ están en el subárbol $\text{izq}(A)$.

Análogamente, todos los elementos $i > r$ están en el subárbol $\text{der}(A)$.

Obs. 2: r es ancestro de todos los elementos $i \neq r$.

Obs. 3: para todo $i < r$, $\text{profundidad}(i, A) = \text{profundidad}(i, \text{izq}(A)) + 1$. Análogo con $i > r$ y der .

ABB óptimo - Costo de un ABB

Podemos reescribir el costo de la siguiente manera:

$$\begin{aligned} \text{costo}(A, f) &= \sum_{i=1}^{|f|} f[i] \cdot \text{profundidad}(i, A) \\ &= \sum_{i=1}^{|f|} f[i] + \sum_{i=1}^{r-1} f[i] \cdot \text{profundidad}(i, \text{izq}(A)) + \sum_{i=r+1}^{|f|} f[i] \cdot \text{profundidad}(i, \text{der}(A)) \end{aligned}$$

Notar que la segunda y tercer sumatoria son similares a la definición original, por lo que podemos substituir por recursión:

$$\text{costo}(A, f) = \sum_{i=1}^{|f|} f[i] + \text{costo}(\text{izq}(A), f[1..(r-1)]) + \text{costo}(\text{der}(A), f[(r+1)..|f|])$$

ABB óptimo - Backtracking

2. Escribir un algoritmo de backtracking que encuentre el ABB óptimo para un f dado.

Idea: debemos elegir una raíz, y después dejamos que recursivamente se decida el subárbol izquierdo y derecho.

$$\text{CostoOptimo}(f) = \begin{cases} 0 & |f| = 0 \\ \sum_{k=1}^{|f|} f(k) + \min_{1 \leq r \leq |f|} \{ \text{CostoOptimo}(f[1..r-1]) + \text{CostoOptimo}(f[r+1..|f|]) \} & \text{caso contrario} \end{cases}$$

Nota de implementación: en general, cuando implementamos este tipo de algoritmos es mejor pasar el array y dos índices *desde* y *hasta* para representar la selección de un subarray.

ABB óptimo - Complejidad

3. Dar una cota superior para la complejidad (Ayuda: pasar de la función recursiva a una recurrencia que solo dependa del tamaño de la entrada).

$$\text{CostoOptimo}(f) = \begin{cases} 0 & |f| = 0 \\ \sum_{k=1}^{|f|} f(k) + \min_{1 \leq r \leq |f|} \{ \text{CostoOptimo}(f[1..(r-1)]) + \text{CostoOptimo}(f[(r+1)..|f|]) \} & \text{caso contrario} \end{cases}$$

La función respeta la siguiente recursión (con $n = |f|$)

$$T(n) = \sum_{k=1}^n (T(k-1) + T(n-k)) + O(n)$$

ABB óptimo - Complejidad

$$T(n) = \sum_{k=1}^n (T(k-1) + T(n-k)) + O(n)$$

Notemos que cada parte de la suma se repite ($k = n - (n - k)$), es decir cada $T(k)$ aparece dos veces, además podemos "abrir" el O con una constante $C > 0$ y tenemos que

$$T(n) = 2 \sum_{k=0}^{n-1} T(k) + Cn$$

ABB óptimo - Complejidad

$$T(n) = 2 \sum_{k=0}^{n-1} T(k) + Cn$$

Ahora si tomamos la recursión con un elemento menos tenemos

$$T(n-1) = 2 \sum_{k=0}^{n-2} T(k) + C(n-1)$$

Restando $T(n) - T(n-1)$, se cancela casi todo y nos queda

$$T(n) - T(n-1) = 2T(n-1) + C$$

Finalmente reordenando

$$T(n) = 3T(n-1) + C$$

ABB óptimo - Complejidad

$$T(n) = 3T(n - 1) + C$$

Es decir, es equivalente a un problema donde tenemos un árbol de recursión ternario completo.

Podemos concluir que $T(n) = O(3^n)$.

ABB óptimo - Demostración

4. Probar que el algoritmo es correcto.

Dado un f queremos mostrar que *CostoOptimo* minimiza la función *costo* sobre el conjunto de soluciones candidatas: todos los ABB posibles con los elementos de $[n]$. Haremos inducción sobre $n = |f|$.

$P(n) = \{ \text{CostoOptimo}(f) = \min_A \text{costo}(A, f), \text{ es decir, } \text{CostoOptimo} \text{ computa el costo del ABB de costo óptimo para todo array } |f| \text{ de tamaño } n \}$.

Caso base ($n=0$): si $n = 0$, el intervalo es vacío. El único ABB posible es el árbol vacío, de costo 0, que es lo que devuelve el algoritmo.

ABB óptimo - Demostración

Caso inductivo: hacemos inducción fuerte, es decir vamos asumir que vale $P(0)$ a $P(n)$ y queremos ver que vale $P(n + 1)$.

- ▶ Todo ABB sobre f tiene alguna raíz $r \in [1, |f|]$.
- ▶ Por la propiedad de ABB, su subárbol izquierdo usa exactamente las claves de $[1, r - 1]$ y su subárbol derecho las de $[r + 1, |f|]$.
- ▶ Por hipótesis inductiva, las llamadas recursivas devuelven los costos de los mejores subárboles posibles para esos dos intervalos.
- ▶ Por lo tanto, para una raíz fija r , el algoritmo obtiene el mejor ABB que tiene raíz r .
- ▶ Como prueba todas las raíces posibles y se queda con el menor costo, obtiene el mejor ABB sobre $|f|$.
- ▶ El costo es el correcto ya que, dados los costos de los mejores subárboles izquierdo y derecho, se aplica la fórmula recursiva de *costo*, sumándoles $\sum_{k=1}^{|f|} f[k]$ a los costos de los subárboles.